



CHRONIQO

DOMÆNEANALYSE

SW7BAC-01 BACHELORPROJEKT

BILAG 2

Elev:

Mads Dittmann Villadsen

202209869 / AU715368

Vejleder:

Tommy Bjerre Nielsen

tbn@ece.au.dk

MAJ 2026

Indholdsfortegnelse

Figurliste	ii
Tabelliste	iii
1. Indledning og formål.....	1
1.1. Formål med domæneanalysen.....	2
1.2. Placering i udviklingsprocessen.....	2
2. Metode	3
2.1. Den anvendte analysemetode.....	4
2.1.1. Identifikation af begreber (navneordsanalyse).....	4
2.1.2. Identifikation af relationer (udsagnsordsanalyse).....	4
2.1.3. Fastsættelse af multipliciteter.....	4
2.1.4. Finpudsning og afgrænsning	5
2.2. Fra use cases til et fælles fagsprog (Ubiquitous Language).....	5
3. Identifikation af begreber (trin 1).....	6
3.1. Bruttoliste via navneordsanalyse	7
3.2. Identificerede koncepter (Conceptual Classes).....	8
3.3. Begreber reduceret til attributter	10
3.4. Fravalgte og systemtekniske begreber	11
4. Identifikation af relationer og multipliciteter (trin 2-3)	12
4.1. Fastlæggelse af domænerelationer	13
4.2. Særlige overvejelser vedrørende kommentarer og tråde.....	16
4.3. Dataintegritet og ejerskab (Cascading)	16
5. Finpudsning og systemafgrænsning (trin 4).....	17
6. Den endelige domæne model (UML)	18
7. Begrebskatalog (Data Dictionary)	19
8. Referencer	22
9. Bilag.....	23

Figurliste

Figur 1: Den endelige konceptuelle domænemodel for Chroniqo	18
---	----

Tabelliste

Tabel 1: Bruttoliste af identificerede navneord	7
Tabel 2: Identificerede domænekoncepter (Conceptual Classes)	8
Tabel 3: Oversigt over domænebegreber reduceret til attributter	10
Tabel 4: Oversigt over fravalgte og systemtekniske begreber	11
Tabel 5: Definition af domænerelationer, multipliciteter og tilhørende domænereregler.....	14
Tabel 6: Data Dictionary for Chroniqo	20

1. Indledning og formål

I dette afsnit introduceres formålet med domæneanalysen for Chroniqo samt analysens placering i det samlede udviklingsforløb. Målet er at etablere en klar forståelse for, hvorfor konceptuel domænemodellering er et kritisk skridt, inden det egentlige softwaredesign og databasearbejde påbegyndes.

1.1. Formål med domæneanalysen

Dette dokument præsenterer domæneanalysen for Chroniqo. Formålet med analysen er at identificere, afgrænse og strukturere de centrale koncepter og begreber, der eksisterer i den virkelighed, systemet skal understøtte.

Chroniqo bevæger sig inden for et domæne centreret omkring kronisk syge, deres dagsform og den asynkrone sociale interaktion. For at kunne bygge et robust softwareprodukt er det essentielt at forstå disse koncepter uafhængigt af den teknologiske implementering. Domænemodellen fungerer derfor som en visuel ordbog for projektet; den etablerer et fælles fagsprog (Ubiquitous Language), der sikrer, at der er en entydig forståelse af, hvad en *Dagsform*, et *Fællesskab* eller et *Opslag* reelt dækker over, samt hvordan disse relaterer sig til hinanden.

1.2. Placering i udviklingsprocessen

I henhold til den anvendte ASE-udviklingsmodel (1) fungerer domæneanalysen som den kritiske bro mellem kravspecifikationen (hvad systemet skal gøre) (2) og det detaljerede systemdesign (hvordan det skal gøres). Analysen tager direkte udgangspunkt i de udarbejdede Epics og User Stories (jf. Bilag 1: Kravspecifikation (2)), men ser bort fra tekniske detaljer som brugergrænseflader (UI) og databaseteknologier for at fastholde fokus på domænets logik. Det resulterende begrebskatalog og UML-klassediagram danner i de efterfølgende faser det direkte fundament for systemets databasemodel (ER-diagram) og den objektorienterede applikationsmodel.

2. Metode

I dette afsnit redegøres for den metodiske tilgang, der ligger til grund for udarbejdelsen af domæneanalysen. Afsnittet beskriver den trinvis proces fra den indledende tekstanalyse af systemkravene til den endelige etablering af et fælles fagsprog, der binder problemdomænet sammen med den tekniske implementering.

2.1. Den anvendte analysemetode

Domæneanalysen er udarbejdet ud fra en struktureret og iterativ proces i fire trin (3), baseret på anerkendte principper for objektorienteret analyse - herunder Craig Larmans retningslinjer for identifikation af Conceptual Classes (4). De følgende underafsnit beskriver hvert af disse trin og deres betydning for modellens udformning.

2.1.1. Identifikation af begreber (navneordsanalyse)

Systemets User Stories analyseres for centrale navneord. Her skelnes der tidligt mellem, hvad der udgør et selvstændigt koncept (en klasse), og hvad der blot beskriver en egenskab ved et koncept (en attribut).

2.1.2. Identifikation af relationer (udsagnsordsanalyse)

Med afsæt i de identificerede begreber findes de indbyrdes sammenhænge gennem en systematisk udsagnsordsanalyse. Ved at nærstudere sprogbrugen i de udarbejdede User Stories og kravspecifikationen identificeres de handlinger, der binder domænets entiteter sammen.

Et eksempel på denne proces er analysen af handlingen: "En Bruger opretter et Opslag i et Fællesskab". Her udledes de logiske associationer, som er fundamentale for systemets funktionalitet. Formålet med dette trin er at transformere statiske begreber til en dynamisk model, der afspejler de faktiske interaktioner og ansvarsområder i Chroniqo. Dette sikrer, at datastrukturen ikke blot opbevarer information, men aktivt understøtter brugernes dagligdag og sociale udveksling.

2.1.3. Fastsættelse af multipliciteter

De logiske sammenhænge, der er beskrevet i kravspecifikationen, oversættes her til præcise mængderelationer (multipliciteter). Det defineres f.eks., hvordan et Opslag er knyttet til et Fællesskab i Brugernes hverdag - herunder om et Opslag overhovedet kan eksistere uden at være en del af et specifikt Fællesskab. Dette sikrer, at datamodellen teknisk understøtter den måde, Brugerne rent faktisk skal interagere på.

2.1.4. Finpudsning og afgrænsning

For at sikre, at modellen er letforståelig for både udviklere og domæneeksperter, fjernes tekniske begreber, der ikke er en del af brugerens virkelighed. Det betyder, at implementeringsdetaljer som *API* og *Token* sorteres fra, så kun de kernebegreber, der er relevante for de kronisk syges interaktioner, står tilbage. Herunder defineres det også, hvordan objekter som f.eks. Opslag og Bruger hænger logisk sammen gennem komposition og aggregering.

2.2. Fra use cases til et fælles fagsprog (Ubiquitous Language)

For at løfte analysen fra en sproglig øvelse til reel arkitektonisk værdiskabelse anvendes principper fra Domain-Driven Design (DDD). Ambitionen er at etablere det tidligere beskrevne fælles fagsprog (Ubiquitous Language) (5), som sikrer, at de begreber, der anvendes i domænemodellen, går igen i både kravspecifikationen, brugergrænsefladen og den efterfølgende tekniske implementering.

Ved konsekvent at anvende de præcise domænetermene, der er identificeret i denne analyse - som f.eks. Dagsform frem for det mere generiske Humør, og Fællesskab frem for Gruppe - på tværs af krav, databasedesign og kildekode, minimeres misforståelser under implementeringen. Dette sikrer, at systemlogikken i koden direkte afspejler den virkelighed, de kronisk syge befinder sig i, hvilket reducerer risikoen for misforståelser i udviklingsprocessen.

Da systemets kildekode og databasestruktur udarbejdes på engelsk, etableres der en fast 1:1-mapping af domænetermene. Dette sikrer, at begreber som Dagsform konsekvent optræder som `DailyStatus`, og Fællesskab som `Community` i koden. Denne systematiske kobling dokumenteres i analysens afsluttende begrebskatalog for at sikre, at domænets fælles fagsprog bevares intakt og entydigt på tværs af sprogene.

3. Identifikation af begreber (trin 1)

I det første trin af domæneanalysen tages der direkte afsæt i systemets Use Cases og kravspecifikation. Formålet er at udtrække centrale navneord, der repræsenterer objekter, koncepter og idéer i systemet.

3.1. Bruttoliste via navneordsanalyse

Gennem en systematisk gennemgang af projektets Epics og User Stories (jf. Bilag 1: Kravspecifikation (2)) blev følgende bruttoliste af navneord identificeret, som vist i Tabel 1:

Tabel 1: Bruttoliste af identificerede navneord baseret på projektets Epics og User Stories fra Kravspecifikationen

Bruttoliste
<i>Bruger, E-mail, Adgangskode, Profil, Diagnose, Lidelser, Dagsform, Note, Kalender, Systembesked, Opslag, Kladde, Fællesskab, Kommentar, Chat, Gruppechat, Besked, Emoji, Administrator, Moderator, Venskab, Venskabsanmodning.</i>

For at skabe en overskuelig og dækkende domænemodel er denne bruttoliste efterfølgende blevet filtreret. Filtreringen tager stilling til, hvorvidt et navneord udgør et selvstændigt koncept (en klasse), en egenskab ved et koncept (en attribut), eller om det ligger uden for domænemodellens rækkevidde.

3.2. Identificerede koncepter (Conceptual Classes)

Følgende navneord i Tabel 2 er vurderet til at besidde en tilstrækkelig kompleksitet, relationel tyngde eller en defineret livscyklus, der begrundet deres status som selvstændige klasser i domæne-modellen. Ved at modellere disse som klasser sikres det, at systemet kan håndtere deres unikke identitet og de handlinger, der knytter sig til dem:

Tabel 2: Identificerede domænekoncepter med 1:1-mapping og ansvarsområder.

Identificerede koncepter (Conceptual Classes)		
Begreb (dansk)	Teknisk navn (engelsk)	Ansvar og karakteristika (rationalisering)
Bruger	User	Systemets centrale entitet. Fungerer som ejer af data og bærer af adgangsrettigheder.
Fællesskab	Community	Organisatorisk enhed, der grupperer brugere og indhold omkring specifikke emner.
Opslag	Post	Primær informationsbærer i et fællesskab med egen livscyklus (opret, rediger, slet).
Kommentar	Comment	En tidsstempelt respons, der er uadskilleligt knyttet til et specifikt opslag.
Dagsform	DailyStatus	Log-entitet, der fastholder brugerens tilstand. Modelleres som klasse pga. historik og unik identitet.
Chat	Conversation	Den logiske ramme og kontekst for asynkron kommunikation mellem brugere.
Besked	Message	Den enkelte meddelelse med specifikt indhold, der sendes i en Conversation.
Venskab	Friendship	Registrerer en etableret, gensidig relation mellem to brugere. Modelleres som klasse, da den har sin egen oprettelsesdato og styrer adgangsrettigheder til privat indhold.

Venskabsan- modning	FriendRequest	En initieret, men endnu ikke bekræftet venskabsforespørgsel. Har sin egen livscyklus (fra PENDING til ACCEPTED / DECLINED) og udgør et selvstændigt transaktionsobjekt.
Kladde	PostDraft	Et ufærdigt opslag under udarbejdelse. Modelleres som selvstændig klasse, da den har en parallel livscyklus til Post, men aldrig publiceres direkte - brugeren beslutter aktivt, om kladden skal offentliggøres.

3.3. Begreber reduceret til attributter

Visse navneord fra analysen betragtes ikke som selvstændige objekter med egen identitet, men fungerer rettere sagt som simple egenskaber eller værdityper (Value Objects). Som det fremgår af Tabel 3, indføres disse i modellen som attributter, da de udelukkende eksisterer for at beskrive en egenskab ved en overordnet klasse:

Tabel 3: Oversigt over domænebegreber reduceret til attributter.

Domænebegreber reduceret til attributter		
Egenskab (attribut)	Tilhører klasse	Beskrivelse og datatype
E-mail / Password	User	Identificerende og sikkerhedsrelaterede værdier (Tekstværdi).
Profilbeskrivelse	User	En valgfri tekstuel præsentation af brugeren (Tekstværdi).
Note	DailyStatus	Supplerende tekst, der uddyber den numeriske dagsform-registrering.
Emoji (Quick-use)	Message	Simpel værditype brugt til hurtig social interaktion uden selvstændig livscyklus
Foretrukne Emojis (Quick-reactions)	User	En liste over brugerens personligt udvalgte standardreaktioner (Liste af tekstværdier / Array of strings).
Diagnose / Lidelser	User	MVP-kontekst modelleret som kategoriserende metadata (Liste af tekstmærkater / tags).
Systembesked	Message	En intern markør, der angiver, om en besked er auto-genereret af systemet (Boolesk værdi / Boolean).

3.4. Fravalgte og systemtekniske begreber

En række navneord fra kravspecifikationen er bevidst udeladt fra den konceptuelle domænemodel. Dette er gjort i overensstemmelse med principperne for domæneanalyse (3, 4, 6) for at isolere tekniske implementeringsdetaljer og dermed fastholde fokus på domænets kerne. De udeladte begreber samt begrundelsen herfor fremgår af nedenstående Tabel 4:

Tabel 4: Oversigt over fravalgte og systemtekniske begreber med tilhørende rationalisering.

Fravalgte og systemtekniske begreber		
Begreb (navneord)	Kategori for udeladelse	Faglig begrundelse (rationalisering)
Administrator / Moderator	Attribut / Rolle	Betragtes som specifikke rettigheder eller roller knyttet til en <code>User</code> . Det er ikke en selvstændig entitet, da de deler kerneegenskaber med standardbrugeren.
Kalender / Oversigt	UI-artefakt (View)	Repræsenterer en visuel præsentation af data (et view). Logikken ligger i filtrering af <code>DailyStatus</code> -objekter og er ikke et selvstændigt domæneobjekt.
Google OAuth / API	Teknisk artefakt	Rene implementeringsdetaljer relateret til systemets infrastruktur og eksterne integrationer. Disse hører til i applikationslaget, ikke i domænemodellen.
Sessioner / Tokens	Tilstandshåndtering	Midlertidige tekniske mekanismer til at håndtere brugeradgang og sikkerhed, som ikke udgør en del af domænets kernekoncepter.

4. Identifikation af relationer og multipliciteter (trin 2-3)

I andet og tredje trin af domæneanalysen fokuseres der på at skabe dynamik i domænemodellen ved at definere, hvordan de identificerede koncepter interagerer med hinanden. Afsnittet fastlægger de logiske associationer, multipliciteter (mængdeforhold) samt de bagvedliggende domænerregler. Herunder præsenteres særlige overvejelser om dataintegritet og de hierarkiske strukturer, der sikrer en korrekt håndtering af systemets indhold.

4.1. Fastlæggelse af domænerelationer

Selve identifikationen af relationerne sker ved at udlede de centrale udsagnsord fra projektets User Stories (2). Disse handlinger oversættes til tekniske associationer, hvorefter de præcise rammer for interaktionen fastlægges via multipliciteter. Dette afklarer f.eks., om en entitet kan eksistere uafhængigt (valgfri relation, $0..1$), kræver en obligatorisk tilknytning (1), eller kan indgå i flere relationer ($0..*$ eller $1..*$).

Baseret på analysen af systemets kernefunktionalitet er følgende centrale relationer og domæneregler i Tabel 5 fastlagt:

Tabel 5: Definition af domænerelationer, multipliciteter og tilhørende domænereregler.

Domænerelationer, multipliciteter og tilhørende domænereregler				
Koncept A	Relation (udsagnsord)	Koncept B	Multiplicitet (A til B)	Faglig begrundelse (domænerregel)
User	registrerer / har	DailyS-tatus	1 til 0..*	En bruger kan have mange dagsforms-registreringer (historik). En dagsform tilhører altid præcis én bruger.
User	er medlem af	Community	0..* til 0..*	En bruger kan være med i mange fællesskaber. Et fællesskab kan have mange brugere.
User	Opretter	Post	1 til 0..*	En bruger kan oprette flere opslag. Et opslag har altid præcis én forfatter.
Community	indeholder	Post	0..1 til 0..*	Et opslag kan tilknyttes et specifikt fællesskab, men behøves det ikke. Et opslag publiceret direkte på brugerens profil har ingen fællesskabstilknytning (communityId er null). Et fællesskab kan have mange opslag.
User	skriver	Comment	1 til 0..*	En bruger kan skrive mange kommentarer. En kommentar har altid én forfatter.

Post	har	Comment	1 til 0..*	En kommentar er altid uløseligt knyttet til et specifikt opslag. Opslaget kan have mange kommentarer.
User	deltager i	Conversation	1..* til 0..*	En bruger kan have mange chats. En chat har altid mindst en (ofte to eller flere) brugere.
Conversation	indeholder	Message	1 til 0..*	En besked eksisterer kun i konteksten af en specifik chat (Conversation).
User	sender	Message	1 til 0..*	En besked har altid præcis én afsender (User).
Comment	svarer på	Comment	0..1 til 0..*	En kommentar kan være et direkte svar til en anden kommentar (nested reply). En kommentar kan have mange svar, men peger kun tilbage på max en forælder-kommentar.
User	indgår i	User (Friendship)	0..* til 0..*	En bruger kan have mange venskaber. Et venskab er gensidigt og involverer altid præcis to brugere.
User	sender / modtager	FriendRequest	1 til 0..*	En bruger kan sende og modtage mange anmodninger. En anmodning har altid præcis én afsender og én modtager.
User	gemmer	Draft	1 til 0..*	En bruger kan have mange kladder. En kladde tilhører altid præcis én forfatter og kan aldrig eksistere uden en ejer.

4.2. Særlige overvejelser vedrørende kommentarer og tråde

Som det fremgår af de definerede relationer i Tabel 5, fungerer et opslag (Post) som en central knudepunkt-klasse. For at imødekomme MVP-kravene om svar med flere tråde (nested replies, jf. US3.12, Bilag 1: Kravspecifikation (2)) er domænemodellen udformet med en association til sig selv (Self-association) på klassen Kommentar (Comment). Dette muliggør en svarer på-relation, hvor en kommentar kan referere til en anden eksisterende kommentar, hvilket teknisk understøtter hierarkiske tråde uden at komplicere den overordnede domænestruktur.

4.3. Dataintegritet og ejerskab (Cascading)

En væsentlig overvejelse i modellen er de stærke afhængigheder mellem brugere og deres data. F.eks. er Opslag (Post) og Kommentarer (Comment) defineret med en obligatorisk relation til en Bruger (User). Dette valg sikrer dataintegritet, da det i systemlogikken fastlægges, at intet indhold kan eksistere uden en identificerbar forfatter.

Desuden fungerer dette som en tidlig forebyggelse af de risici for dataintegritetsfejl og forældreløse data, der blev identificeret i projektets risikoanalyse (jf. Risiko R7, Bilag 3: Risikoanalyse (7)). Rent teknisk betyder det, at systemets design tvinger en stillingtagen ved sletning af en bruger: Skal tilknyttet indhold slettes via Cascade Delete, eller skal det bevares i anonymiseret form?

5. Finpudsning og systemafgrænsning (trin 4)

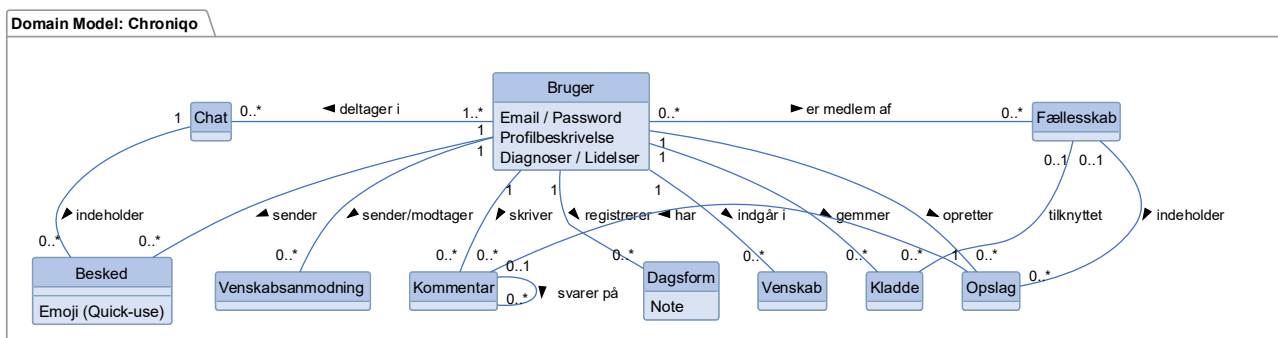
Som det fremgår af afsnit 3.4, er modellen iterativt blevet rensset for tekniske implementeringsdetaljer (f.eks. API'er, views og auth-mekanismer) (3). Dette sikrer, at den resulterende domænemodel fremstår som et rent konceptuelt overblik, der udelukkende repræsenterer de logiske sammenhænge i Chroniqos domæne.

I finpudsningen er der ligeledes taget stilling til komposition; det er vurderet, at et Opslag og en Kommentar ikke kan eksistere uafhængigt af en Bruger, hvilket etablerer et stærkt hierarkisk ejerskab i systemet.

Den endelige afgrænsning af domænet er visualiseret i det efterfølgende UML-diagram, hvor udelukkende de domæne-centrale koncepter samt deres indbyrdes associationer og multipliciteter er repræsenteret.

6. Den endelige domæne model (UML)

Nedenstående UML-klassediagram udgør den visuelle repræsentation af Chroniqos domæne. I overensstemmelse med principperne for domæneanalyse (3, 6) udelades metoder (operations) og specifikke datatyper på attributterne for at fastholde fokus på det konceptuelle niveau. Læsepile på associationerne indikerer, i hvilken retning domænelogikken og de indbyrdes sammenhænge skal fortolkes.



Figur 1: Den endelige konceptuelle domæne model for Chroniqo repræsenteret som UML-klassediagram.

7. Begrebskatalog (Data Dictionary)

For at fastholde det etablerede fælles fagsprog (5) på tværs af projektets faser, fungerer Tabel 6 som systemets endelige begrebskatalog. Her fastlægges den direkte oversættelse (mapping) mellem domænets danske domæneterner og de engelske termer, som konsekvent anvendes i systemets kildekode og databasedesign. Dette sikrer en entydig sporbarhed fra de indledende krav til den endelige tekniske implementering:

Tabel 6: Data Dictionary for Chroniqo - Systematisk mapping af domænetermene til teknisk terminologi.

Begrebskatalog (Data Dictionary)			
Domænebegreb (dansk)	Implementeringsterm (engelsk)	Systemkontekst og definition	Primære attributter
Bruger	User	Platformens centrale aktør, som ejer data og interagerer med andre brugere.	email, hashedPassword, bio, conditions
Dagsform	DailyStatus	En historisk log-entitet, der indikerer brugerens aktuelle funktions- og energiniveau på en given dag.	note, date, value
Fællesskab	Community	En tematisk gruppe, hvor brugere kan samles og udveksle opslag.	
Opslag	Post	En asynkron informationsdeling (tekst / medie) oprettet i et fællesskab.	
Kommentar	Comment	Et svar på et opslag eller en underkommentar (tråd).	
Chat	Conversation	Den logiske beholder for en 1:1- eller gruppesamtale mellem brugere uden for de åbne fællesskaber.	
Besked	Message	En konkret, tidsstempelt meddelelse sendt i en Chat.	content
Venskab	Friendship	En etableret, gensidig social relation mellem to brugere på platformen. Låser op for privat indhold og beskedrettigheder afhængig af brugerindstillinger.	createdAt

Venskabsanmodning	FriendRequest	En initieret anmodning om at indgå i et venskab. Eksisterer i en midlertidig tilstand, indtil den accepteres eller afvises af modtageren.	status, createdAt
Kladde	PostDraft	Et ufærdigt opslag gemt lokalt til brugerens profil eller et fællesskab. Deler struktur med Opslag, men publiceres ikke, før brugeren aktivt vælger det.	title, type, content, metadata

8. Referencer

1. Gregersen T. Vejledning til udfærdigelse af projektrapporter - version 1.5. Vejledningsdokument. Ingeniørhøjskolen Aarhus Universitet; 20. november 2018.
3. Jakobsen FB. Domænemodeller (Version 1.0) / Systemdomæneanalyse og Domænemodeller. Forelæsningsslides. Ingeniørhøjskolen Aarhus Universitet; [år ukendt].
4. Larman C. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development. 3rd ed: Prentice Hall; 2004.
5. Fowler M. Ubiquitous Language [Internet]. 31 October 2006. [hentet 27. januar 2026]. Tilgængelig fra: <https://martinfowler.com/bliki/UbiquitousLanguage.html>.
6. Fowler M. Domain Model [Internet]. 05 March 2003. [hentet 27. januar 2026]. Tilgængelig fra: <https://martinfowler.com/eaCatalog/domainModel.html>.

9. Bilag

2. Villadsen MD. Chroniqo: Kravspecifikation. Bilag 1. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
7. Villadsen MD. Chroniqo: Risikoanalyse. Bilag 3. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.